

## Question 1: Code Tracing - Branching, Loops, and Functions

What is the output of the following code segment? Include all outputs printed to the console in your answer.

```
def mystery(num):
    total = 0
    for i in range(1, num + 1):
        if i % 2 == 0:
            total += i
        elif i % 3 == 0:
            total += 2*i
        else:
            total -= i
        print("step", i, ":", total)
    return total

def create_list(lst):
    new_list = []
    for item in lst:
        if item % 3 == 0:
            new_list.append(item * 2)
        else:
            new_list.append(item + 1)
    return new_list

x = mystery(4)
y = create_list([x, 3, 5])
print("x:", x)
print("y:", y)
```

## Question 2: Programming - Permutations and exception handling

Given two strings, write a function that checks if one is a permutation of another.

Refer to this definition of a string permutation: A permutation of a string is another string that contains the exact same characters as the original string, but in a different order. The key characteristic is that all characters from the original string must be present in the permutation, and each character must appear the same number of times in both the original and the permuted string. Ensure to handle any exception in your program using exception handlers.

### Example 1:

Input: s = "abcde", t = "cdbae"

Output: True

### Example 2:

Input: s = "abcde", t = "cdbaef"

Output: False

### Example 3:

Input: s = "apple", t = "aleap"

Output: False

Implement the below function:

```
def isPermutation(s: str, t: str) -> bool:
    # code goes here
```

```
#sample calls
print(isPermutation("listen", "silent"))
print(isPermutation("hello", "world"))
print(isPermutation(None, "hello"))
```

### Question 3: Code Tracing - OOP

Take the following code segment:

```
class Shape:
    def __init__(self, height):
        self.height = height

    def calcArea(self):
        return 0

class Trapezoid(Shape):
    def __init__(self, height, base1, base2):
        super().__init__(height)
        self.__base1 = base1
        self.__base2 = base2

    def calcArea(self):
        a = (self.__base1 + self.__base2) // 2
        return a * self.height

    def getPerimeter(self, s1, s2):
        return self.__base1 + self.__base2 + s1 + s2

    def __str__(self):
        return (f"Trapezoid: height: {self.height}, "
                f"base1: {self.__base1}, base2: {self.__base2}")

class Rectangle(Shape):
    def __init__(self, height, width):
        super().__init__(height)
        self.width = width

    def calcArea(self):
        pass

    def __str__(self):
        return f"Rectangle: height: {self.height}, width: {self.width}"
```

```

def main(self):
    tr = Trapezoid(5.5, 7, 9)
    rect = Rectangle(4, 6)

    shapes = [tr, rect]

    # Polymorphism occurs in this loop
    for shape in shapes:
        print(shape)
        print(f"area: {shape.calcArea()}")

    tr.height = 8
    print(f"new height: {tr.height}")
    print(f"new area: {tr.calcArea()}")

    print(f"base1: {tr.__base1}")

    tr.base1 = 14
    print(tr)

if __name__ == "__main__" :
    main ()

```

1. What is the exact output produced before the program stops?
2. Does an error occur? If so, identify the line, state the error type, and explain why it occurs.
3. In the following loop, which `calcArea()` method is called for each object?  

```

for shape in shapes:
    print(shape)
    print(f"area: {shape.calcArea()}")

```
4. Explain how the loop demonstrates polymorphism.

#### **Q4: Conceptual Understanding - Data Structures**

Kevin and Bob have decided, with absolutely no hesitation, to make an AI B2B Agentic SAAS business that shows a person's phone number and their associated name (they don't know contacts exist). This is amazing, and both are excited. However, they aren't really technical, so they decide to hire the entire CS313E class to work on their product (you don't get a share in the company though). They have several problems they want you to solve.

(a) Phone numbers are stored as integers within their system, and names are stored as strings. The issue is having these two connected to one another in some capacity. What data structure can we use to store both phone numbers and names in a relational manner? Defend your answer with at least 1 reason.

(b) The team has decided that they receive phone numbers that aren't unique at times, as users will input the same phone number to commit fraud. What data structure can we use to store only unique numbers, WITHOUT a relation? Defend your answer with at least 1 reason.

## Q5: Code-Writing

You are given a 2D list, where each row represents a student's grades and each column represents their grades in each of these three subjects: Mathematics, Science, and English. For example:

```
grades = [  
    [85, 90, 88], # Student 1's grades in Math, Science, and English  
    [78, 82, 80], # Student 2  
    [90, 85, 92], # Student 3  
    [70, 75, 68]  # Student 4  
]
```

Write a Python program to perform the following three tasks as separate functions:

1. `calculate_averages`: computes the average grade for each student rounded to two decimal places. This function accepts a 2D list of grades and returns a list (of averages).
2. `find_highest_grades`: computes the highest grade in each subject (Math, Science, English) across all of the students' grades for those subjects. This function accepts a 2D list of grades and returns a tuple (of highest grades).
3. `get_unique_grades`: identify the grades across all students and subjects that appear at least once in the entire 2D list of grades. This function accepts a 2D list of grades and returns a set (of unique grades).

Additionally, write a main function that creates the 2D list, calls the above three functions and outputs the averages, highest grades, and unique grades in the following format:

```
Averages: [87.67, 80.0, 89.0, 71.0]  
Highest Grades: (90, 90, 92)  
Unique Grades: {85, 90, 88, 78, 82, 80, 92, 70, 75, 68}
```

Rules and notes:

1. All three functions **must** accept parameters and have return types as specified. You are not required to accept input from the user and can create any arbitrary 2D list (or use the one provided above).
2. Lambda expression has not been taught in class and **should not** be used.
3. You may use lists, sets, or tuples for intermediate steps inside the functions.
4. You can use any functions provided in the reference sheet.
5. Remember to call `main()` at the end.

## Q6: Code-Tracing

What is the output?

```
import math

def func_a(num):
    return math.sqrt(num) + 1

def func_b(num, i):
    return math.pow(num,i) + math.exp(num)

def func_c(num, i):
    print(f"{{(math.ceil(num)) + (math.floor(i))}}")

def main():
    num_list = [5, 4, 0]
    for i in range(len(num_list)):
        if num_list[i] % 4:
            num_list[i] = 2 * func_a(num_list[i])
        elif num_list[i] % 5:
            num_list[i] = func_a(num_list[i]) + func_b(num_list[i], i)
        else:
            func_c(num_list[i], num_list[i])

    print(str(num_list))

main()
```

### Question 7: Code-Tracing - Tuples

Print the output of the code. If any errors, write the line # it occurs on, and assumes errors are independent of code logic (the code will still finish even with errors).

```
def compare_players(players, favorites):
    common = ()

    for i in range(len(players)):
        for j in range(len(favorites)):
            if players[i] == favorites[j]:
                if players[i] not in common:
                    common += (players[i],)

    print("Common players:", common)
    print("Number of common players:", len(common))

    for player in common:
        print(player, "is at tuple index", players.index(player))

    return common

players = ("Jokic", "SGA", "Giannis", "Tatum", "Lebron")
favorites = ["Curry", "Jokic", "Tatum", "Jokic", "Durant"]

print(players[0])
print(players[2])
print(players[-1])

print("Jokic count:", players.count("Jokic"))
print("Tatum index:", players.index("Tatum"))

result = compare_players(players, favorites)

players[1] = "Curry"
print("Done")
```

## Question 8: Code-Writing - Lists and exceptions

Write a function `calculate_sum` that takes two **lists** of integers, `a` and `b`, and calculates the **sum of the products** of elements in `a` and `b` with the **same index**. Do not add a sum if the elements are identical. Print the sum once it has been calculated. Assume `a` and `b` are the same length.

Note:

- You do not need to take in input for `a` and `b`. Assume they are already defined.
- The function you create must be called in `main()`, and `main()` must be called in the program. Again, no input is needed for `a` and `b`. You are free to use them.
- **Do not** use any functions from the `math` module for implementation.
- Ensure to handle any exception in your program using exception handler.

## Question 9: Code-Tracing - 2D List Operations

Write the output of the following program

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
for i in range(len(matrix)):
    for j in range(len(matrix[i])):
        if (i + j) % 2 == 0:
            matrix[i][j] *= 2
```

```
matrix[0][2] = 4
```

```
matrix[2][1] = 6
```

```
matrix[1][1] = matrix[0][1] + matrix[0][2]
```

```
for row in matrix:
    print(row)
```

What is the output of the following program? Recall that printing lists in Python automatically includes brackets and commas. Hint: Write/type out this problem by hand to see the updates!

## Q10: Code-Writing - Clean and process the sentence and exceptions

Write a function `clean_sentence(sentence, details_requested)` that:

1. Splits the sentence into parts using `split()`.
2. Examines every character in each part.
3. Keeps only alphabetic characters in each cleaned word.
4. Converts cleaned words to lowercase.
5. Stores removed characters, such as punctuation and digits, in a separate list.
6. Prints the removed characters only when `details_requested` is `True`.
7. Returns the list of cleaned words.

### Q11: What is the output?

```
import string

def build_inverted_index(words):
    index = {}

    for i in range(len(words)):
        word = words[i]

        if word not in index:
            index[word] = []

        index[word].append(i)

    return index

def find_positions(words, target):
    indexes = build_inverted_index(words)
    result = indexes.get(target)

    if not result:
        return False

    return result

sentence = "hello!!! world, 123 coffee? yes: coffee world... hello"
words = sentence.split()
for i, _ in enumerate(words):
    words[i] = words[i].strip(string.punctuation)

print("coffee:", find_positions(words, "coffee"))
print("world:", find_positions(words, "world"))
print("python:", find_positions(words, "python"))
```

## Question 12: Programming

### Zoo and Animal Class Coding Question

Create a program that models animals and a zoo using classes.

#### Part 1: `Animal` class

Define a class called `Animal` with the following:

Private/Public Attributes

- `name` — the animal's name
- `species` — the type of animal
- `zoo` — the name of the zoo the animal belongs to, initially an empty string
- `__happiness` — a private attribute starting at 50

Methods

- `__init__(self, name, species)`
  - Initializes the animal's name, species, zoo, and happiness.
- `play(self)`
  - Multiplies the animal's happiness by 1.5
  - Returns the updated happiness
- `__str__(self)`
  - If the animal is not in a zoo, return:
    - `"{name} is a {species}"`
  - If the animal is in a zoo, return:
    - `"{name} is a {species} in {zoo}"`

#### Part 2: `Zoo` class

Define a class called `Zoo` with the following:

Attributes

- `name` — the zoo's name
- `animals` — a list of `Animal` objects in the zoo

Methods

- `__init__(self, name, animals)`
  - Initializes the zoo's name and list of animals
  - For every animal already in the list, update that animal's `zoo` attribute to the zoo's name
- `add_animal(self, animal)`
  - Adds the given animal to the zoo's list
  - Updates that animal's `zoo` attribute to the zoo's name
- `__str__(self)`
  - Returns a string in this format:

ZooName:

animal1 description  
animal2 description